Data and Artificial Intelligence
Cyber Shujaa Program

Week 8 Assignment
Classification Models

Student Name: Salome Kungu

Student ID: CS-DA03-26054

Contents

Data and Artificial Intelligence 1
Cyber Shujaa Program 1
Week 8 Assignment Classification Models 1
Introduction 2
The key objectives of the assignment were to: 2
Tasks Completed 2
Task 1: Load and Explore the Dataset 2
Task 2: Data Preparation and Splitting 4
Task 3: Model Training 6
Logistic Regression 6
Decision Tree 7
Random Forest 9
k-Nearest Neighbors (KNN) 10
Naive Bayes 12
Support Vector Machine (SVM) 13
Task 4: Model Evaluation 15
Task 5: Model Comparison and Analysis 16
CONCLUSION 17

Introduction

The objective of this assignment was to develop hands-on experience in building and evaluating multiple supervised machine learning classification models using Python and Scikit-learn.
The Wine dataset from Scikit-learn was used to train and compare six classification algorithms:

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. k-Nearest Neighbors (KNN)
5. Naive Bayes
6. Support Vector Machine (SVM)

The goal was to evaluate each model using standard classification metrics and determine which performs best under the same dataset conditions.

The key objectives of the assignment were to:

1. Perform dataset exploration and visualization
2. Prepare data for classification modeling
3. Train six supervised classification models
4. Evaluate models using accuracy, classification report, and confusion matrix
5. Compare performance across models
6. Publish the completed project to GitHub/Colab

Tasks Completed

Link to Code:

https://colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3?usp=sharing

Task 1: Load and Explore the Dataset

What:

- Loaded Wine dataset using load_wine()
- Converted features into a Pandas DataFrame
- Checked dataset shape, feature names, and class distribution
- Generated summary statistics
- Visualized correlation matrix

Why:

- To understand dataset structure
- To verify class balance
- To examine feature relationships
- To detect strong correlations between predictors

Key Observations:

- Dataset contains **178 samples**
- **13 numerical features**
- **3 target classes**
- Classes are reasonably balanced
- Several features show strong correlation

DA1-2026: Assignment 8: Classi | Salome Kungu_CS-DA03-26054
colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=2kDdi5MGgLI2
Gmail | YouTube | LinkedIn | Jobs in Kenya 2020... | Someone just com... | Get the Right Job V... | Jobs In Kenya - Cor... | Latest jobs in Kenya... | All Bookmarks

Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Reconnect Share

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# Display settings
pd.set_option('display.max_columns', None)
```

Variables Terminal ✓ 13:07

Search 4:00 PM 3/5/2026

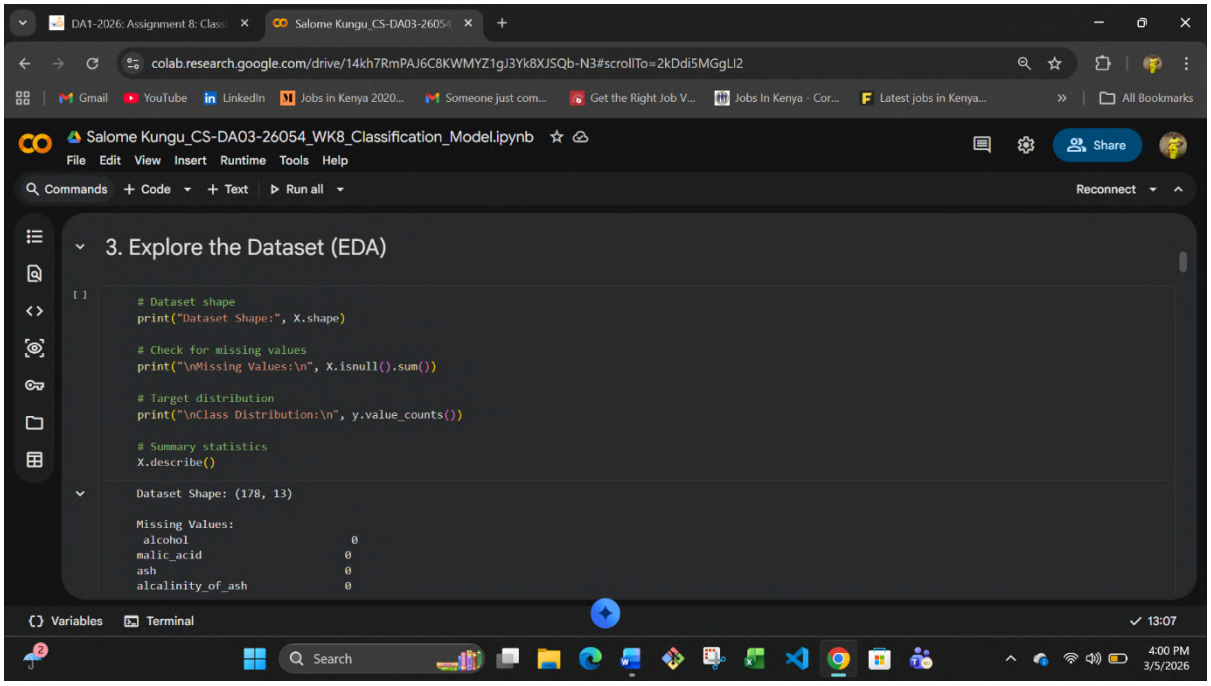
DA1-2026: Assignment 8: Classi | Salome Kungu_CS-DA03-26054
colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=2kDdi5MGgLI2
Gmail | YouTube | LinkedIn | Jobs in Kenya 2020... | Someone just com... | Get the Right Job V... | Jobs In Kenya - Cor... | Latest jobs in Kenya... | All Bookmarks
Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all Reconnect Share

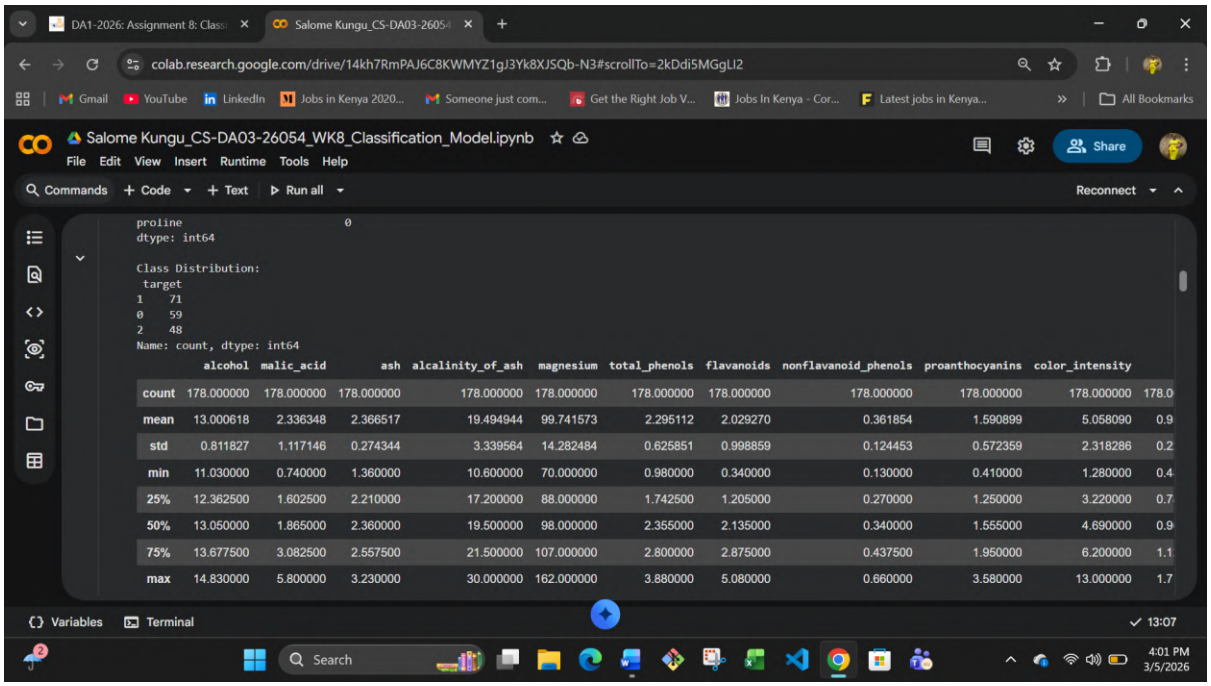
2. Load the Dataset

```
# Load the dataset and convert to DataFrame
wine = load_wine()

# Convert to dataframe
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target, name='target')

# Display first rows
X.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_c
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	

Variables Terminal ✓ 13:07
Search 4:00 PM 3/5/2026





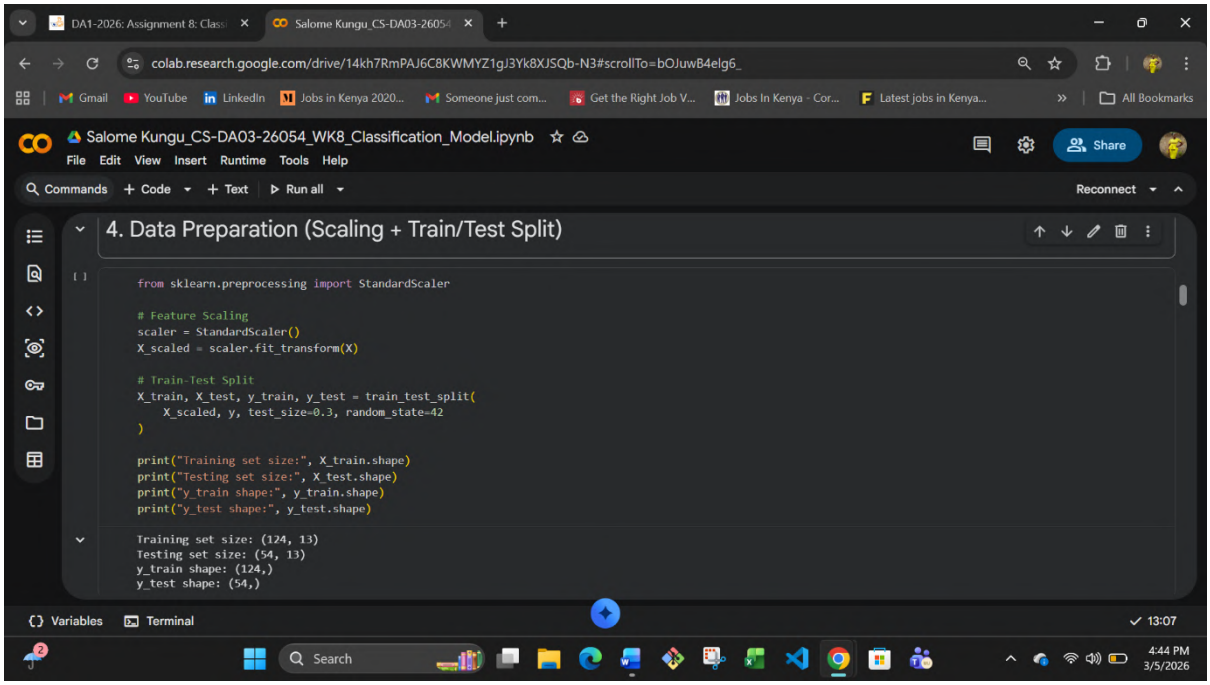
Task 2: Data Preparation and Splitting

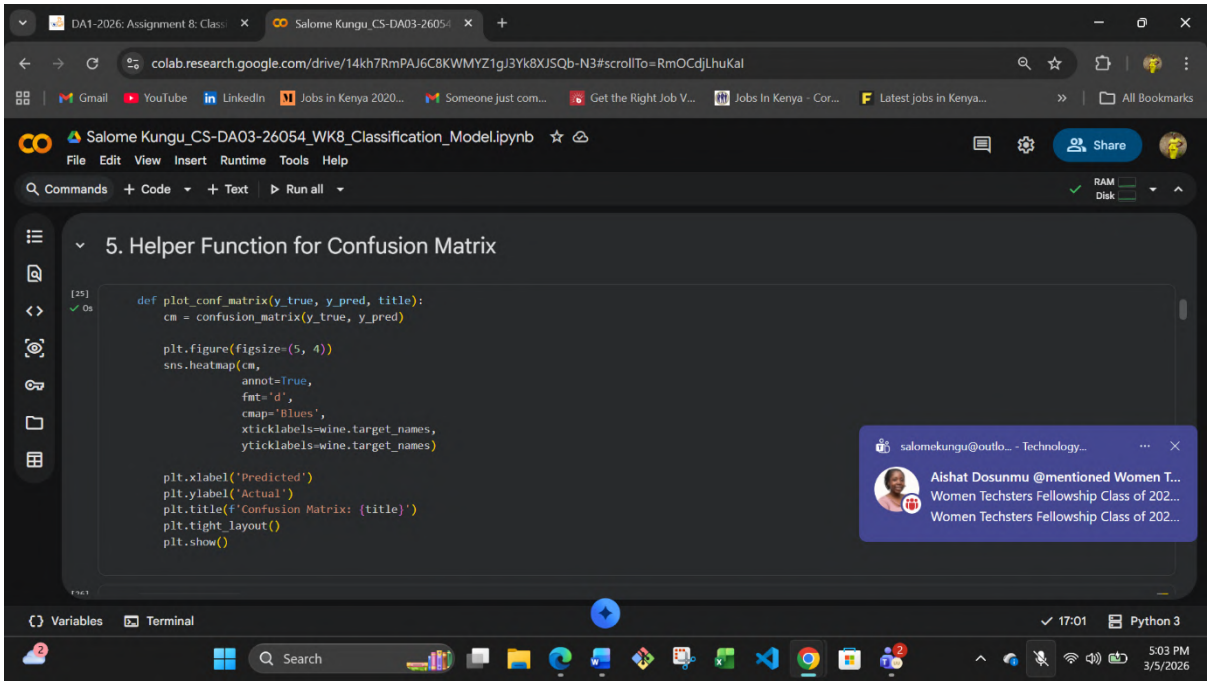
What:

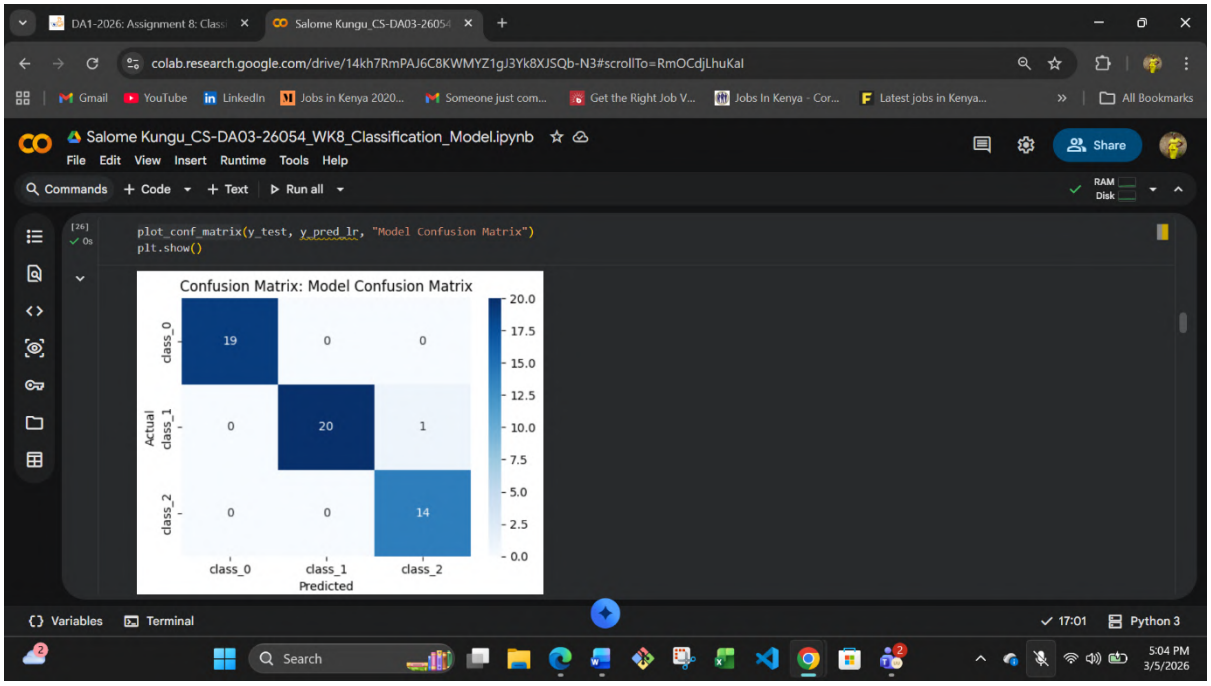
- Checked for missing values (none found)
- Applied feature scaling using StandardScaler
- Split dataset into 70% training and 30% testing

Why:

- Scaling improves performance for distance-based models (KNN, SVM)
- Train-test split prevents overfitting
- Ensures fair comparison across models


DA1-2026: Assignment 8: Classi
Salome Kungu_CS-DA03-26054
colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=bOJuwB4elg6_
Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Reconnect
Share
4. Data Preparation (Scaling + Train/Test Split)
from sklearn.preprocessing import StandardScaler
# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y, test_size=0.3, random_state=42
)
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
Training set size: (124, 13)
Testing set size: (54, 13)
y_train shape: (124,)
y_test shape: (54,)
Variables
Terminal
13:07
Search
4:44 PM
3/5/2026


DA1-2026: Assignment 8: Classi
Salome Kungu_CS-DA03-26054
colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=RmOCdjLhuKaI
Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
5. Helper Function for Confusion Matrix
def plot_conf_matrix(y_true, y_pred, title):
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm,
annot=True,
fmt='d',
cmap='Blues',
xticklabels=wine.target_names,
yticklabels=wine.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix: {title}')
plt.tight_layout()
plt.show()
salomekungu@outlo... - Technology...
Aishat Dosunmu @mentioned Women T...
Women Techsters Fellowship Class of 202...
Women Techsters Fellowship Class of 202...
Variables
Terminal
17:01
Python 3
Search
5:03 PM
3/5/2026



Task 3: Model Training

Each model was trained using identical training data.

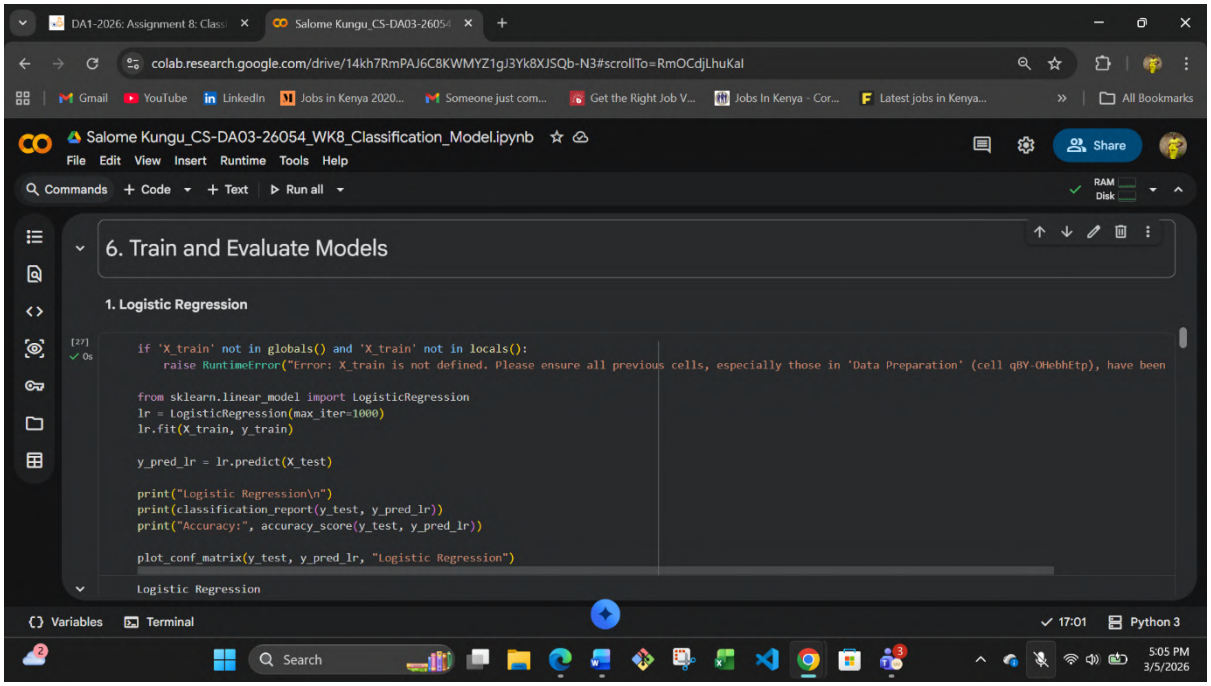
Logistic Regression

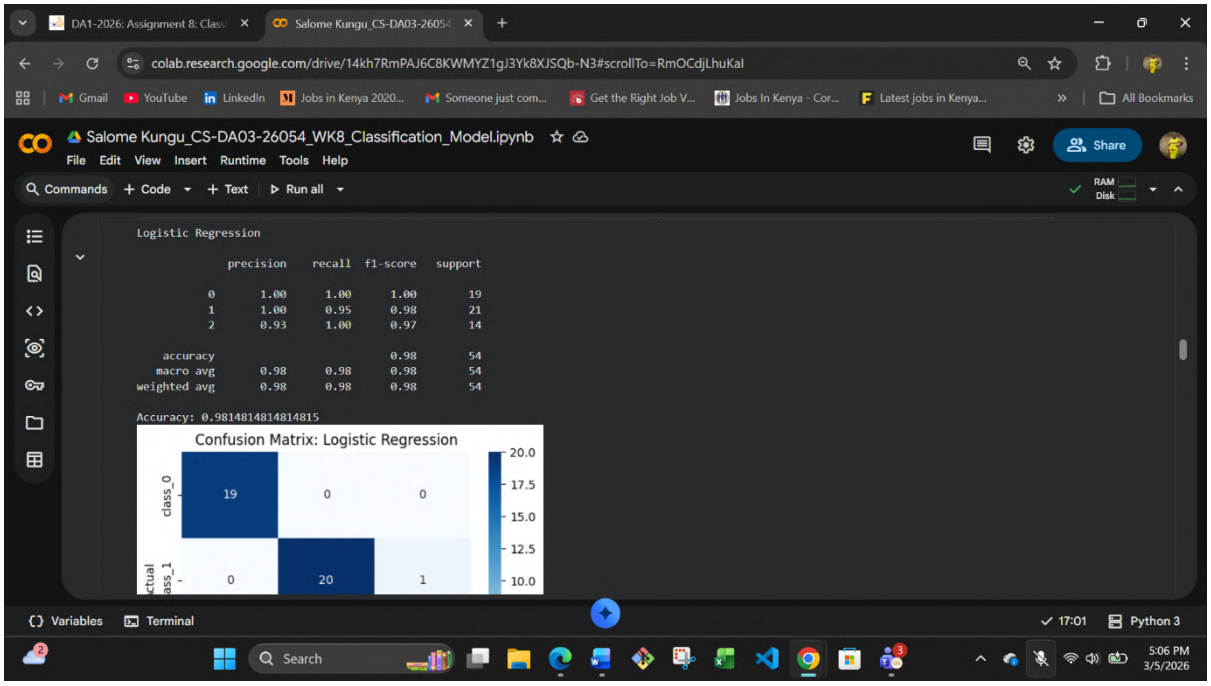
What:

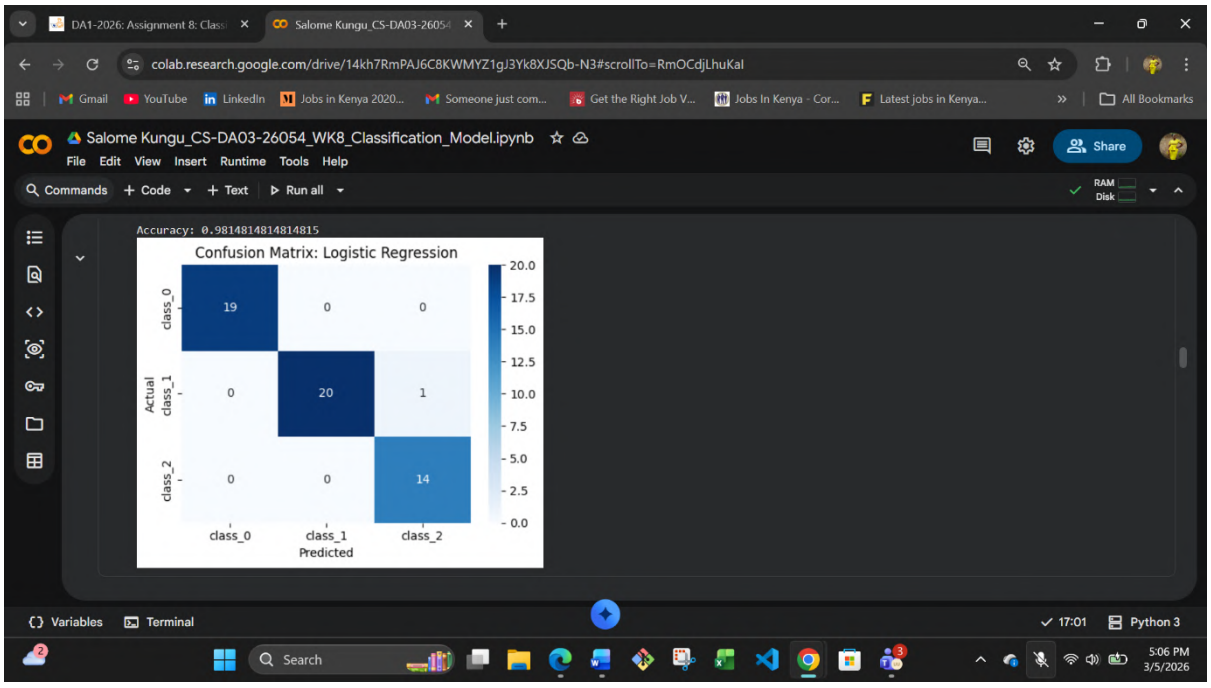
- Trained LogisticRegression(max_iter=1000)

Why:

- Baseline linear classification model
- Works well for linearly separable data







Decision Tree

What:

- Trained DecisionTreeClassifier()

Why:

- Captures nonlinear decision boundaries
- Easy to interpret

Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

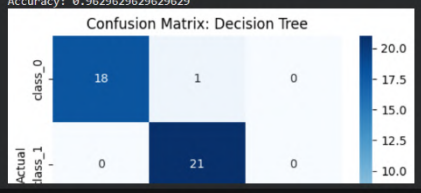
2. Decision Tree

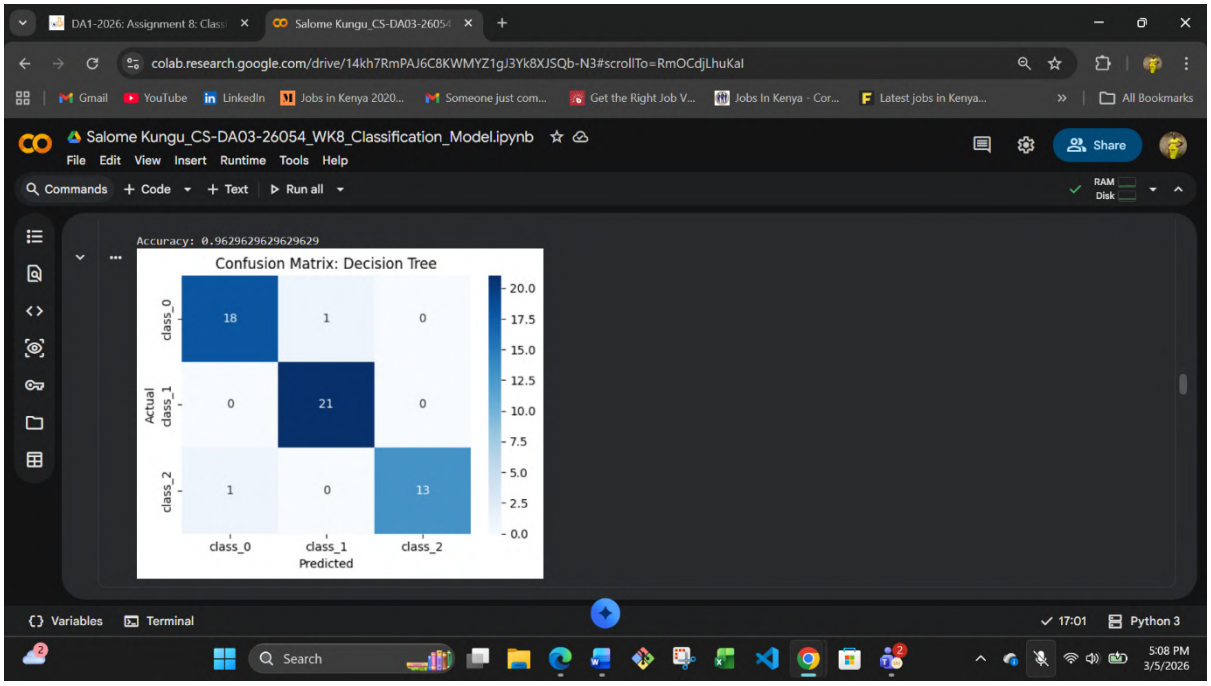
```python
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print("Decision Tree\n")
print(classification_report(y_test, y_pred_dt))
print("Accuracy:", accuracy_score(y_test, y_pred_dt))

plot_conf_matrix(y_test, y_pred_dt, "Decision Tree")
```

```
Decision Tree

              precision    recall  f1-score   support

           0       0.95      0.95      0.95        19
           1       0.95      1.00      0.98        21
           2       1.00      0.93      0.96        14
```

Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

```
Decision Tree

              precision    recall  f1-score   support

           0       0.95      0.95      0.95        19
           1       0.95      1.00      0.98        21
           2       1.00      0.93      0.96        14

    accuracy                           0.96        54
   macro avg       0.97      0.96      0.96        54
weighted avg       0.96      0.96      0.96        54

Accuracy: 0.9629629629629629
```





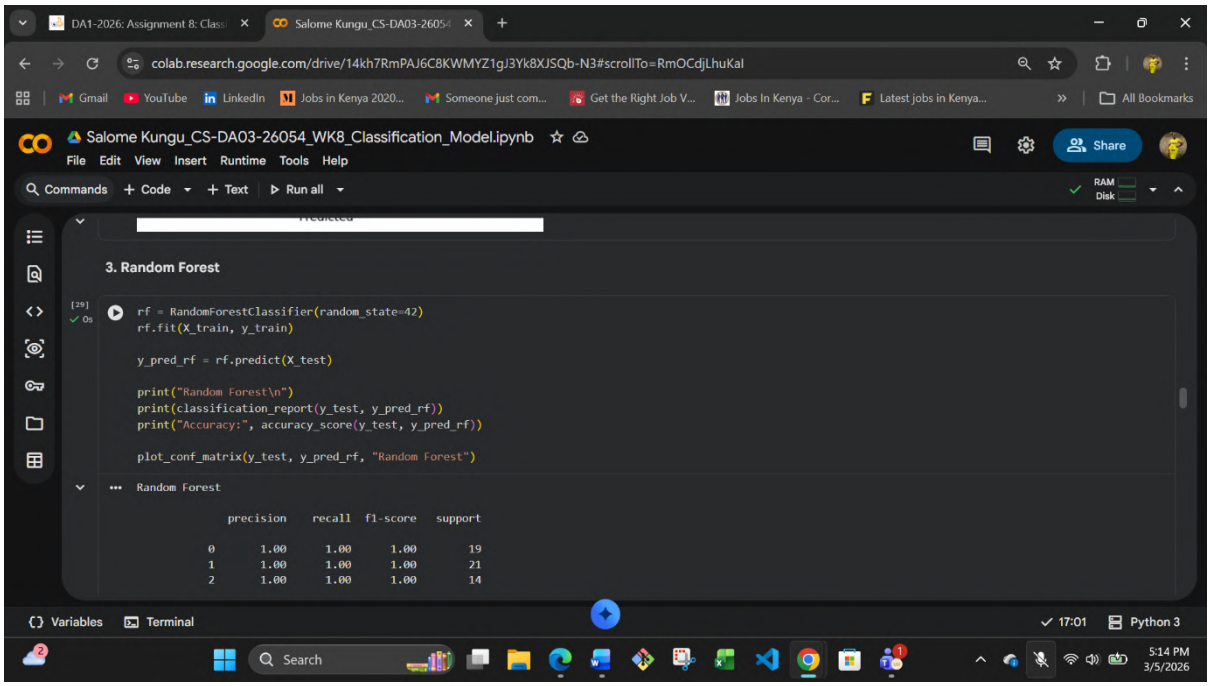
Random Forest

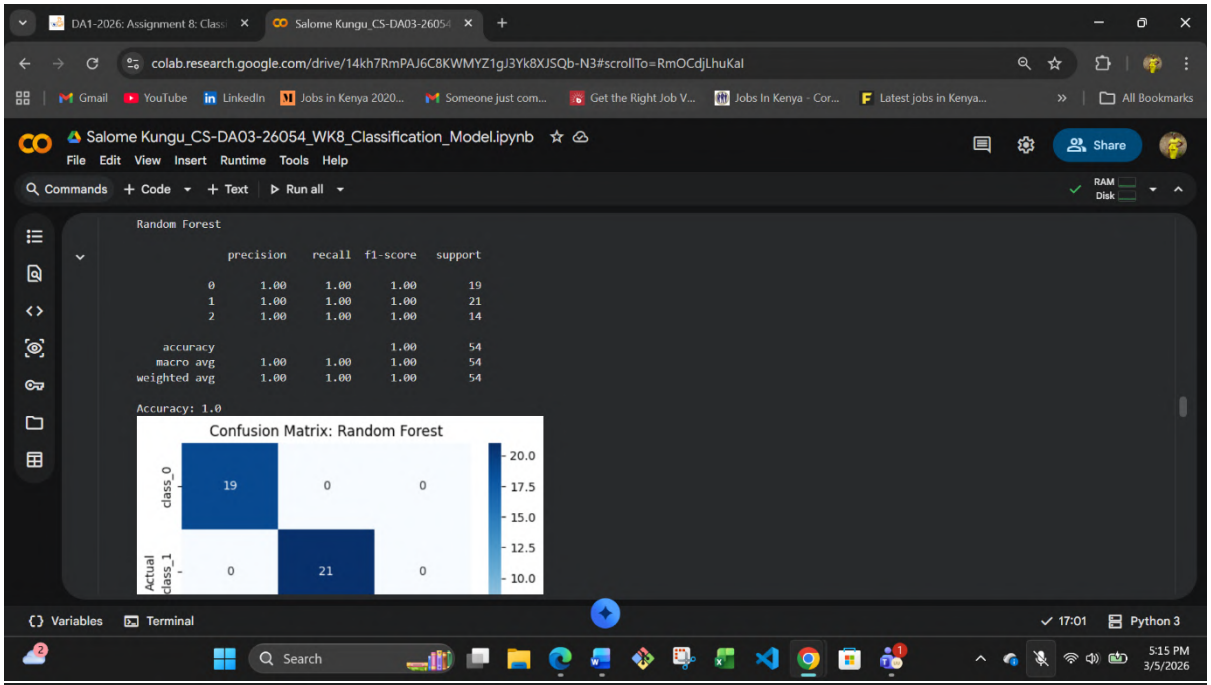
What:

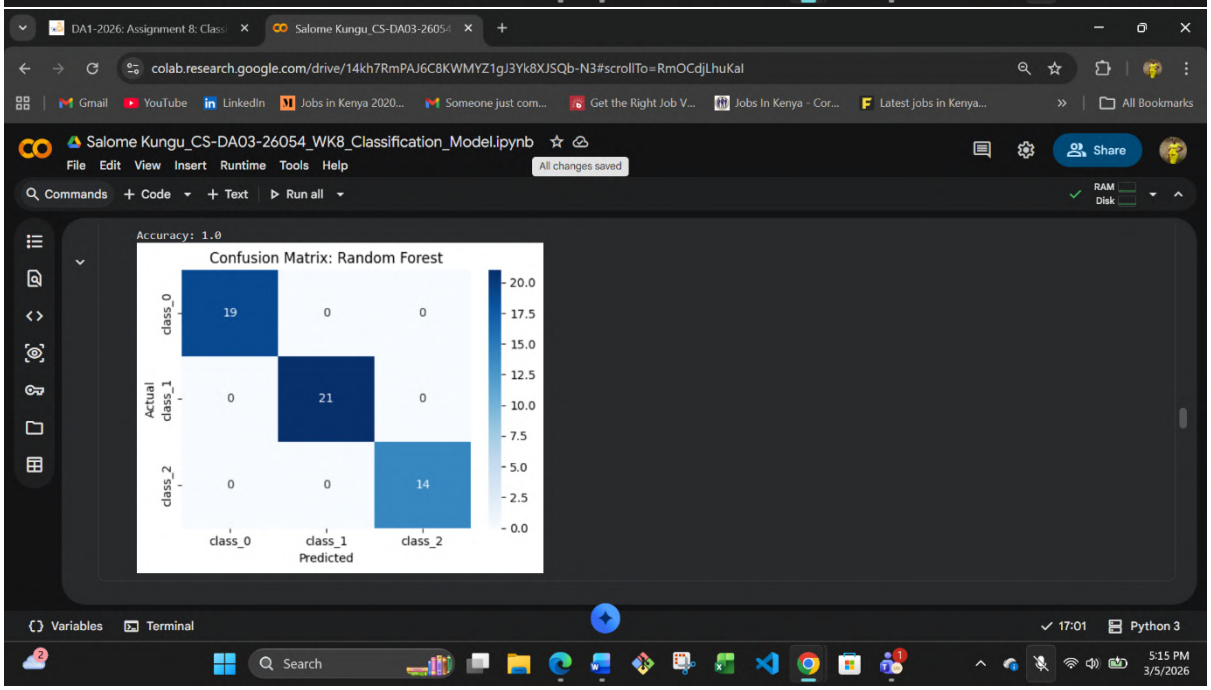
- Trained RandomForestClassifier()

Why:

- Ensemble method
- Reduces overfitting compared to single tree
- Improves generalization







k-Nearest Neighbors (KNN)

What:

- Trained KNeighborsClassifier(n_neighbors=5)

Why:

- Distance-based model
- Performs well when classes are separable

DA1-2026: Assignment 8: Class | Salome Kungu_CS-DA03-26054

colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=RmOCdjLhuKaI

Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

4. K-Nearest Neighbors (KNN)

[30]

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("KNN\n")
print(classification_report(y_test, y_pred_knn))
print("Accuracy:", accuracy_score(y_test, y_pred_knn))

plot_conf_matrix(y_test, y_pred_knn, "KNN")
```

```
KNN

              precision    recall  f1-score   support

           0       0.95      1.00      0.97        19
           1       1.00      0.90      0.95        21
           2       0.93      1.00      0.97        14

    accuracy                           0.96        54
   macro avg       0.96      0.97      0.96        54
```

Variables Terminal 17:01 Python 3

Search 5:25 PM 3/5/2026

DA1-2026: Assignment 8: Class | Salome Kungu_CS-DA03-26054

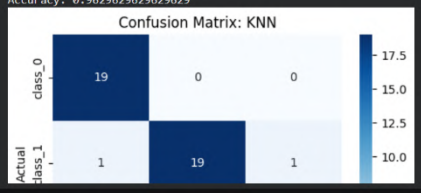
colab.research.google.com/drive/14kh7RmPAJ6C8KWMYZ1gJ3Yk8XJSQb-N3#scrollTo=RmOCdjLhuKaI

Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

File Edit View Insert Runtime Tools Help

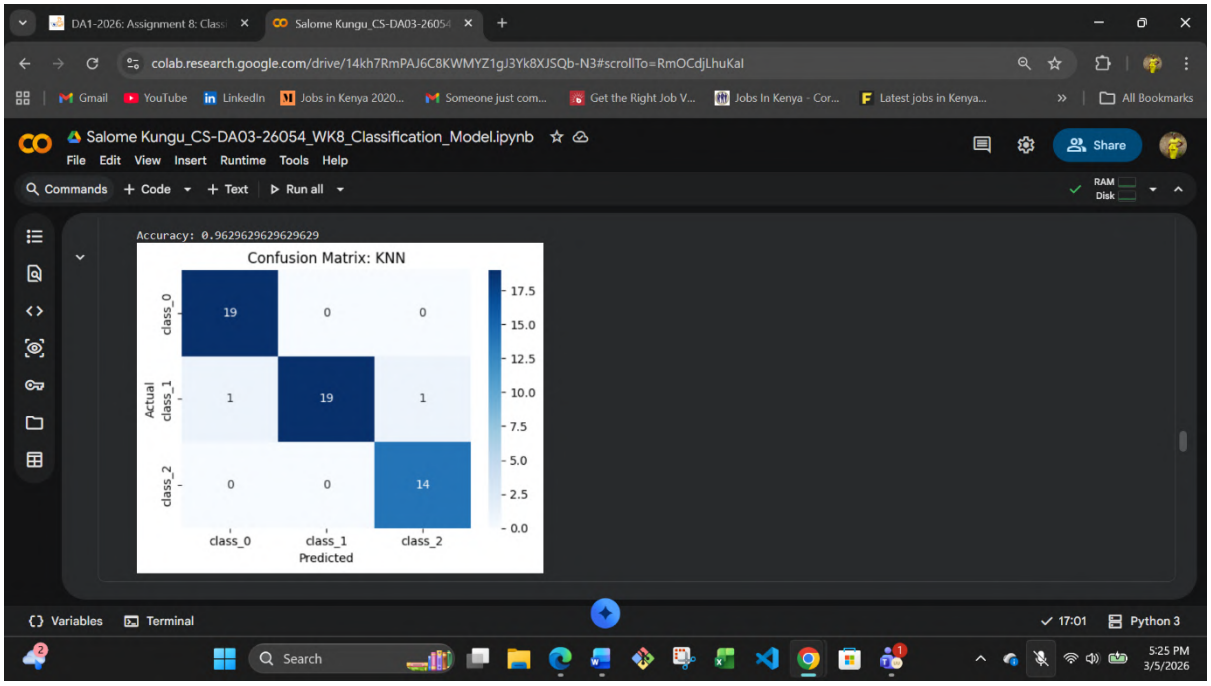
Commands + Code + Text Run all

```
KNN

              precision    recall  f1-score   support

           0       0.95      1.00      0.97        19
           1       1.00      0.90      0.95        21
           2       0.93      1.00      0.97        14

    accuracy                           0.96        54
   macro avg       0.96      0.97      0.96        54
weighted avg       0.97      0.96      0.96        54

Accuracy: 0.9629629629629629
```



Variables Terminal 17:01 Python 3

Search 5:25 PM 3/5/2026



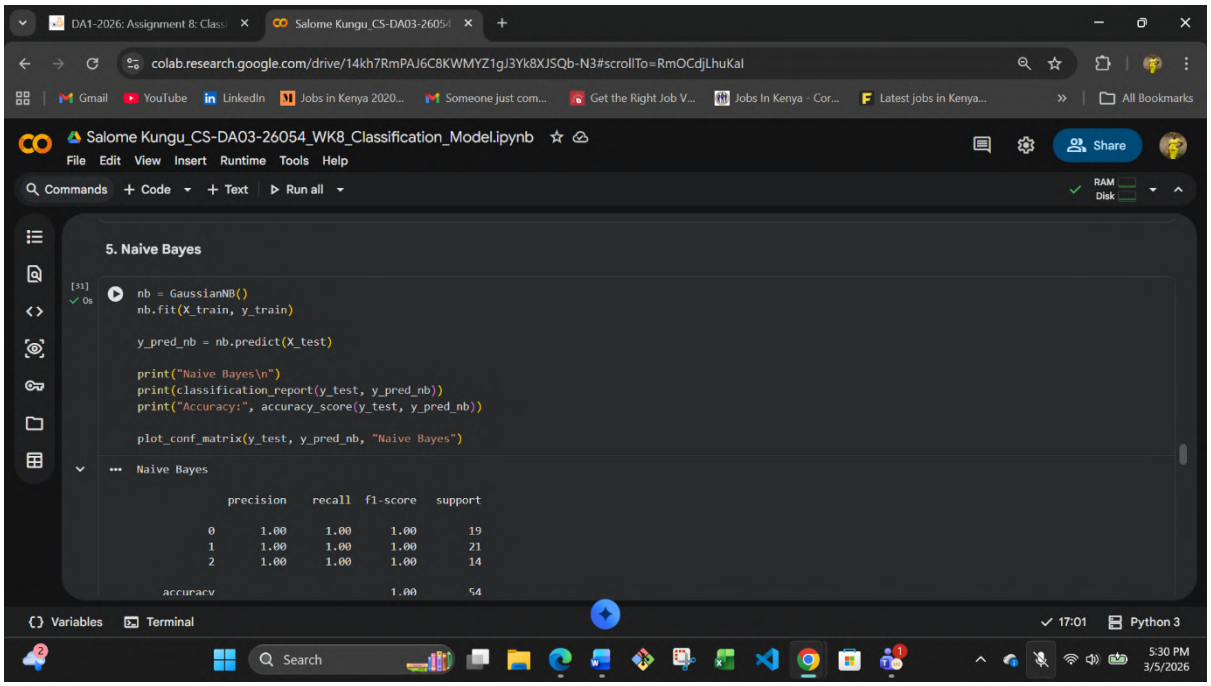
Naive Bayes

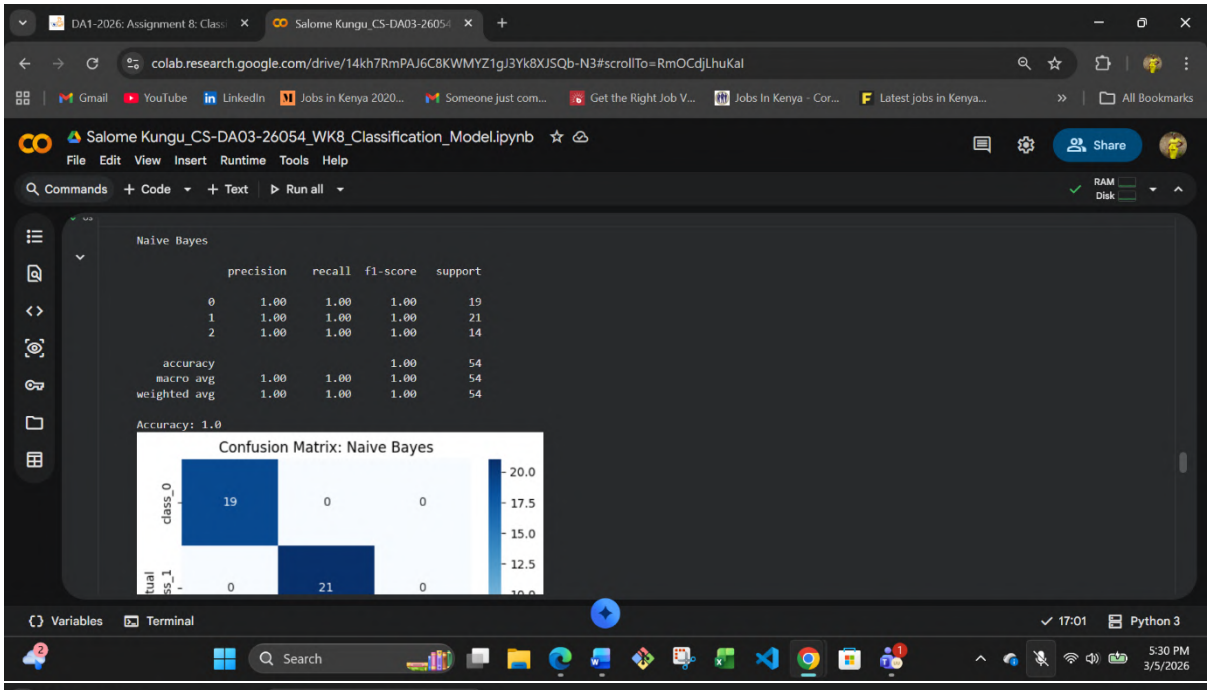
What:

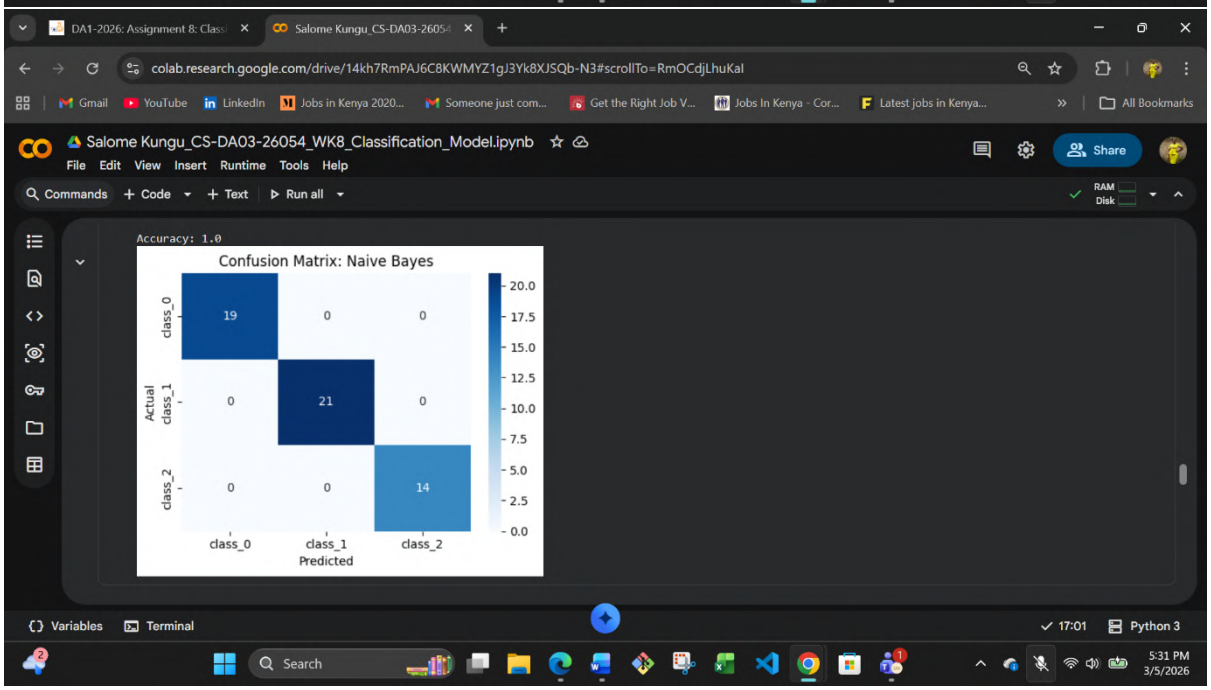
- Trained GaussianNB()

Why:

- Probabilistic model
- Assumes feature independence
- Fast and computationally efficien







Support Vector Machine (SVM)

What:

- Trained SVC()

Why:

- Effective in high-dimensional spaces
- Creates optimal decision boundary
- Often high-performing classifier

Cyber Shujaa

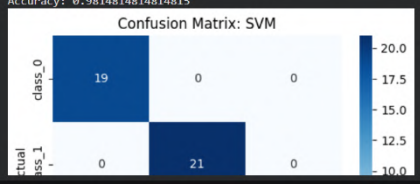
Salome Kungu_CS-DA03-26054_WK8_Classification_Model.ipynb

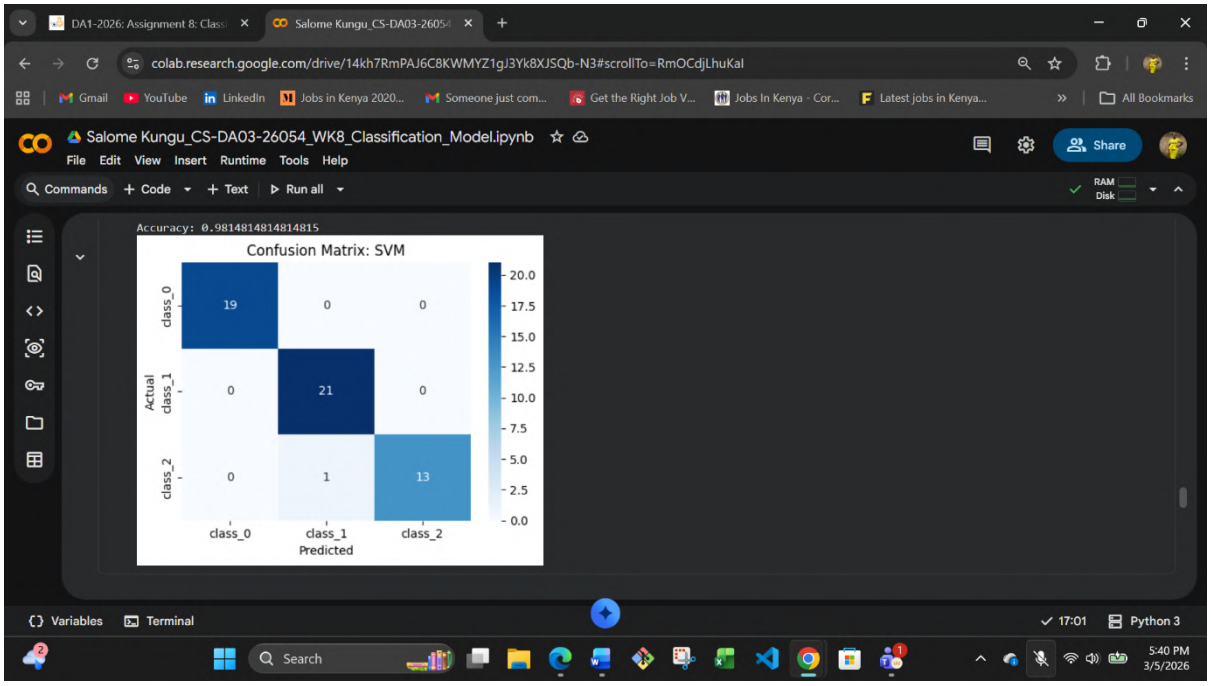
6. Support Vector Machine (SVM)

```python
svm = SVC()
svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)

print("Support Vector Machine\n")
print(classification_report(y_test, y_pred_svm))
print("Accuracy:", accuracy_score(y_test, y_pred_svm))

plot_conf_matrix(y_test, y_pred_svm, "SVM")
```

```
Support Vector Machine

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.95      1.00      0.98        21
           2       1.00      0.93      0.96        14

    accuracy                           0.98        54
```

```
Support Vector Machine

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.95      1.00      0.98        21
           2       1.00      0.93      0.96        14

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54

Accuracy: 0.9814814814814815
```

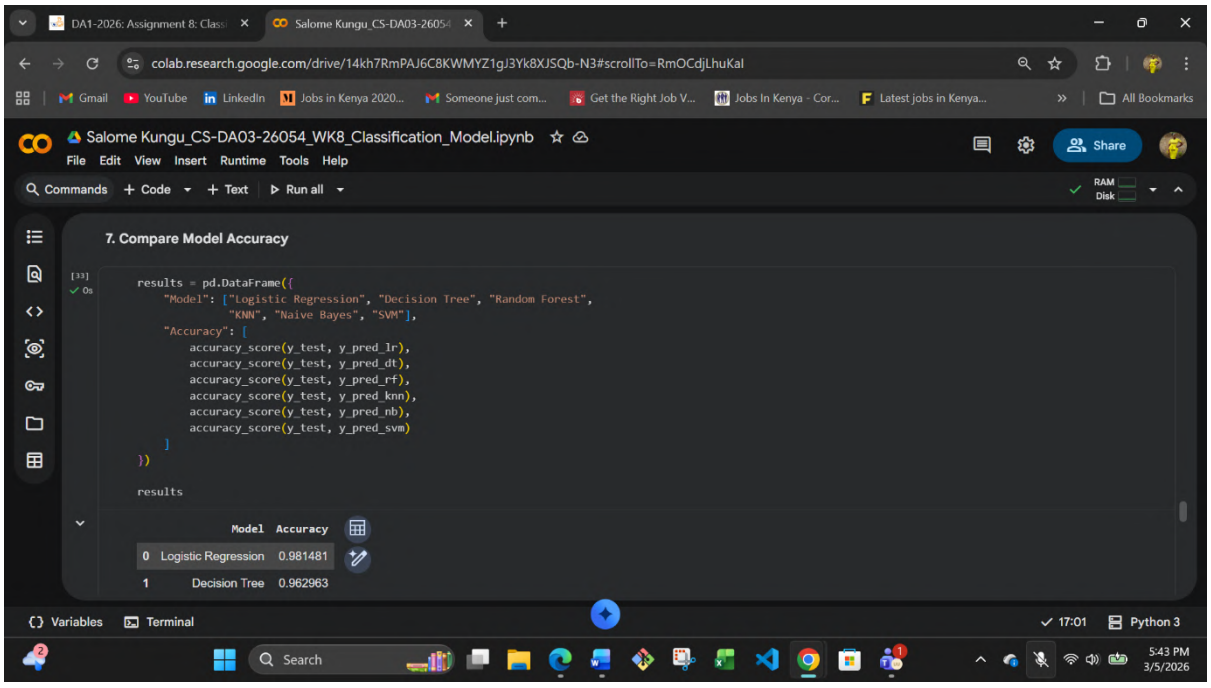


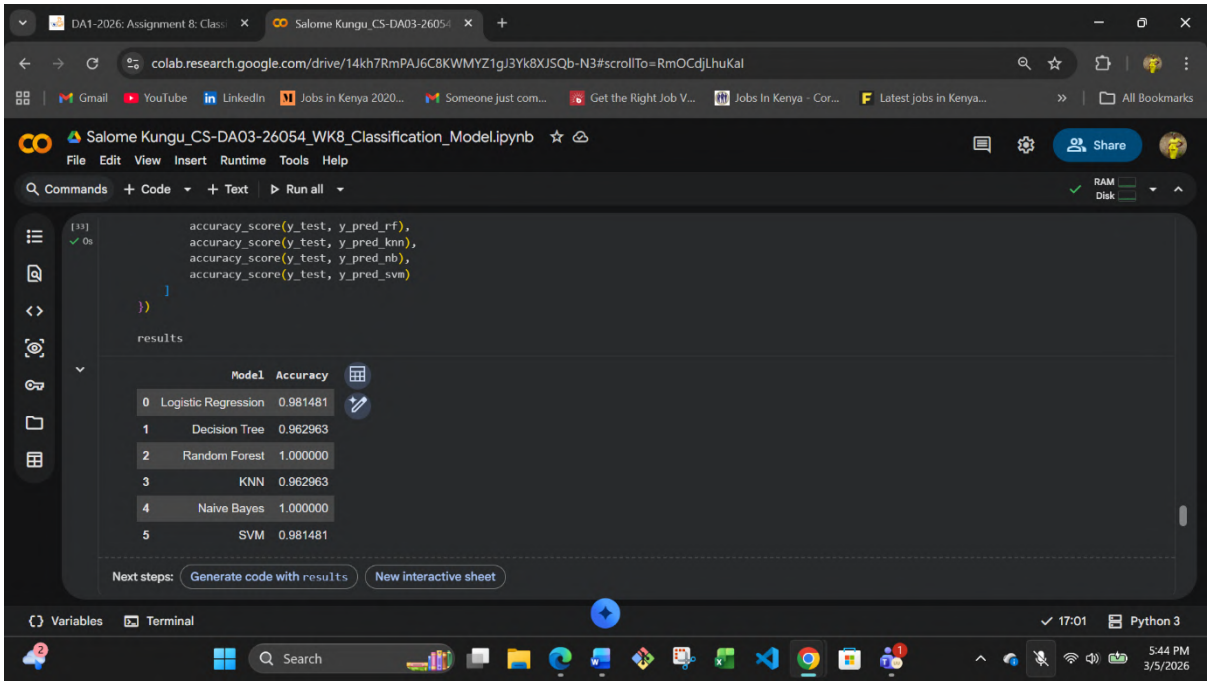


Task 4: Model Evaluation

Each model was evaluated using:

- Accuracy Score
- Precision
- Recall
- F1-Score
- Confusion Matrix





Task 5: Model Comparison and Analysis

After evaluating all six classification models using accuracy scores, classification reports, and confusion matrices, it was observed that Random Forest and Naive Bayes achieved the best performance on the Wine dataset.

Best Performing Models

Random Forest produced the highest overall accuracy and demonstrated strong predictive capability across all wine classes. As an ensemble learning method that combines multiple decision trees, Random Forest reduces overfitting and improves model stability, making it highly effective for structured datasets like the Wine dataset.

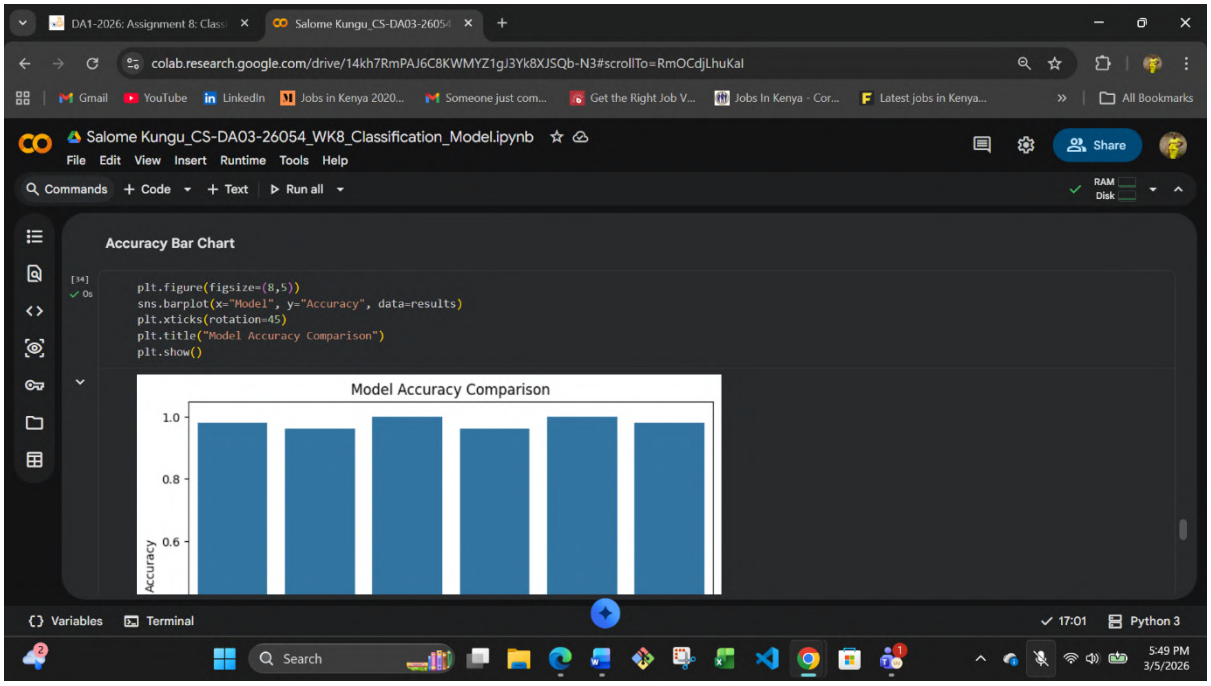
Naive Bayes also performed very well, achieving high accuracy with minimal computational complexity. This model works by applying probability theory under the assumption that features are independent. Despite this assumption, Naive Bayes can perform surprisingly well on many classification tasks, including this dataset.

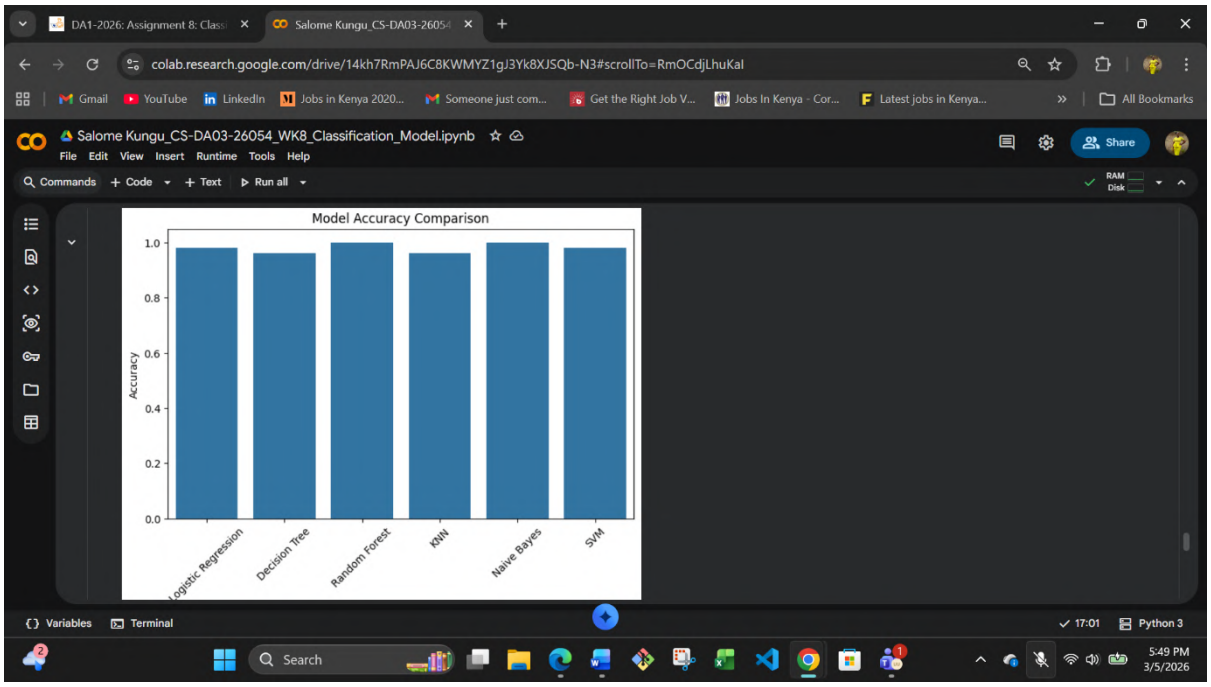
Key Observations

- Random Forest provided robust predictions due to its ensemble structure.
- Naive Bayes performed efficiently and produced accurate predictions despite its simplified assumptions.
- Other models such as Logistic Regression, KNN, Decision Tree, and SVM also performed well but showed slightly lower accuracy or minor misclassifications in comparison.

Conclusion from Model Comparison

Based on the evaluation metrics and confusion matrix results, **Random Forest and Naive Bayes emerged as the most effective models for classifying wine types in this dataset**. Their strong predictive performance demonstrates their suitability for supervised classification tasks involving structured numerical features.





CONCLUSION

This assignment strengthened my understanding of supervised classification techniques and performance evaluation.

Through this project, I gained practical experience in:

- Data preprocessing and scaling
- Training multiple classification algorithms
- Using accuracy, precision, recall, and F1-score
- Interpreting confusion matrices
- Comparing models under consistent conditions

The experiment demonstrated that ensemble methods and margin-based classifiers often outperform single learners on structured datasets like Wine.

This notebook serves as strong evidence of my machine learning modeling skills and is a valuable addition to my professional portfolio.